

Autoencoders

Trabajo Práctico N°5

Ardenghi, Filippo (64306)

Barnatán, Martín Alejandro (64463)

Pedemonte Berthoud, Ignacio (64908)

Testoni, Ezequiel (64709)

1Q 2026

Tabla de Contenidos

01

AE simple:
Problema y Dataset

02

Arquitectura y
Entrenamiento

03

Espacio latente y
Generación

04

Denoising AE

05

Variational AE

06

Conclusiones

01

AE simple: Objetivo y Dataset

Dataset: font.h

01. Objetivo y Dataset

Especificaciones del Dataset

- 32 Caracteres ASCII
- Matriz 5×7
- Arquitectura del Autoencoder

Objetivo de Reconstrucción

Comprimir cada glifo a un **espacio latente de 2D** manteniendo la fidelidad con **≤ 1 píxel de error**.

35

px entrada

2

latente

35

px salida

02

Arquitectura y Entrenamiento

Salida Sigmoide para Píxeles Binarios

02. Arquitectura y Entrenamiento

Representación de Glifos

Los glifos son binarios: cada uno de los 35 píxeles vale 0 o 1.



Función Sigmoid

Acota la salida a $(0,1)$, permitiendo interpretarla como una **probabilidad de píxel encendido**. Encaja perfectamente con el rango del target binario.



Linear

No acota la salida (puede dar < 0 o > 1), perdiendo el sentido probabilístico necesario.



Tanh

Centrada en $(-1,1)$. Requeriría un reescalado forzado de los targets para ser compatible.

Función de Pérdida

02. Arquitectura y Entrenamiento

Comparativa de Gradientes: MSE vs BCE

MSE + sigmoide

$$\frac{\partial L}{\partial z} = (\hat{y} - y) \sigma'(z)$$
$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

Se anula cuando la salida satura (tiende a 0 o 1): el gradiente desaparece.

BCE + sigmoide

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

No hay factor que se anule: corrige aunque la neurona esté saturada.

MSE vs BCE · misma red 35-20-2

mejor max_pixel_error · restarts ≤ 1 px

MSE

10 px

BCE

0 px

Arquitectura Candidata

02. Arquitectura y Entrenamiento



```
{
  "architecture": {
    "encoder_layers": [ 35, 20, 2 ],
    "activation": "tanh",
    "output_activation": "sigmoid",
    "init": "xavier_normal",
    "latent_activation": "linear"
  },
  "training": {
    "optimizer": "adam",
    "loss": "bce",
    "epochs": 15000,
    "lr": 0.001,
    "restarts": 12,
  }
}
```


Con Mayor Profundidad

02. Arquitectura y Entrenamiento



11/12

Éxito $\leq$ 1 px

10/12

0 px

0,25

max_px medio

```
{
  "architecture": {
    "encoder_layers": [ 35, 25, 15, 8, 2 ],
    "activation": "tanh",
    "output_activation": "sigmoid",
    "init": "xavier_normal",
    "latent_activation": "linear"
  },
  "training": {
    "optimizer": "adam",
    "loss": "bce",
    "epochs": 15000,
    "lr": 0.001,
    "restarts": 12,
  },
}
```

Con Aumento de LR y Scheduler

02. Arquitectura y Entrenamiento



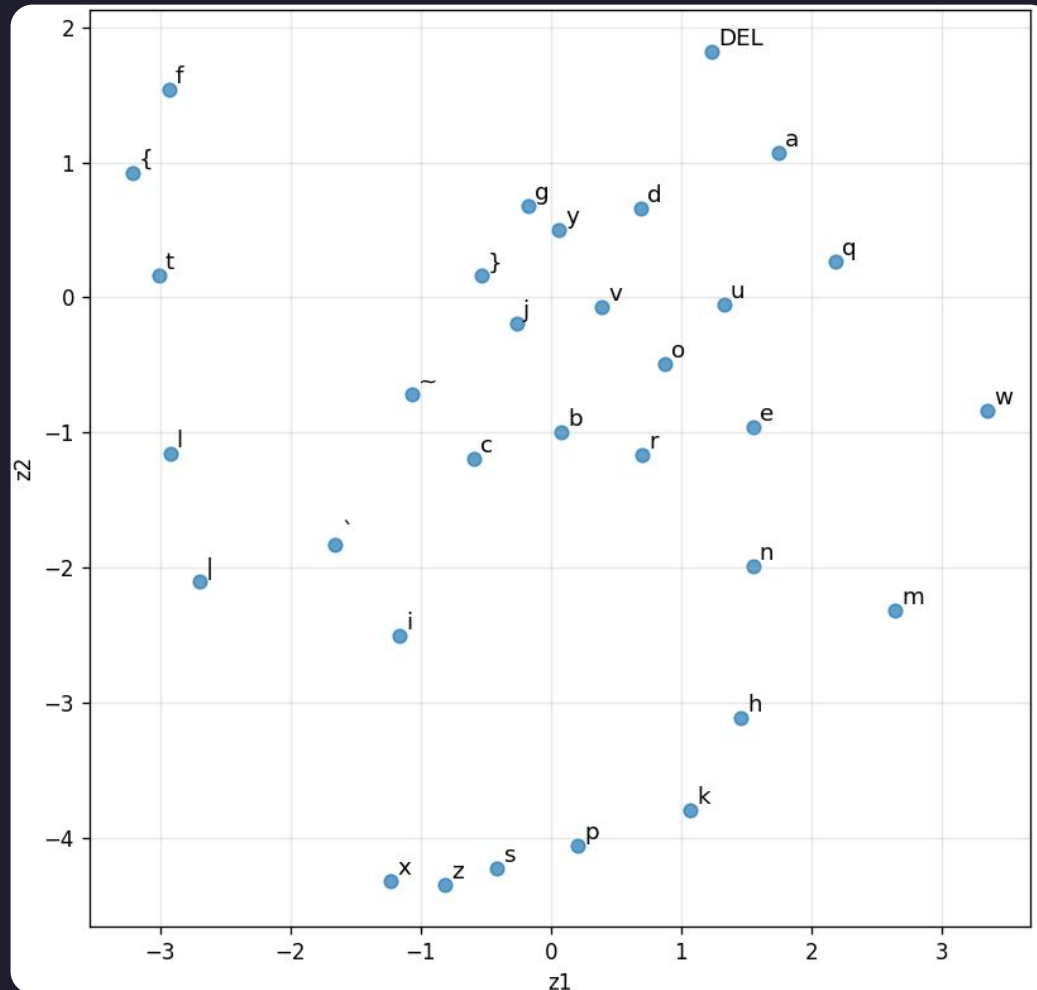
```
{
  "architecture": {
    "encoder_layers": [ 35, 25, 15, 8, 2 ],
    "activation": "tanh",
    "output_activation": "sigmoid",
    "init": "xavier_normal",
    "latent_activation": "linear"
  },
  "training": {
    "optimizer": "adam",
    "loss": "bce",
    "epochs": 15000,
    "lr": 0.003,
    "restarts": 30,
    "lr_schedule": "cosine",
    "lr_min": 0.0
  },
}
```

03

Espacio Latente y Generación

Espacio Latente 2D

03. Espacio Latente y Generación



Reconstrucción

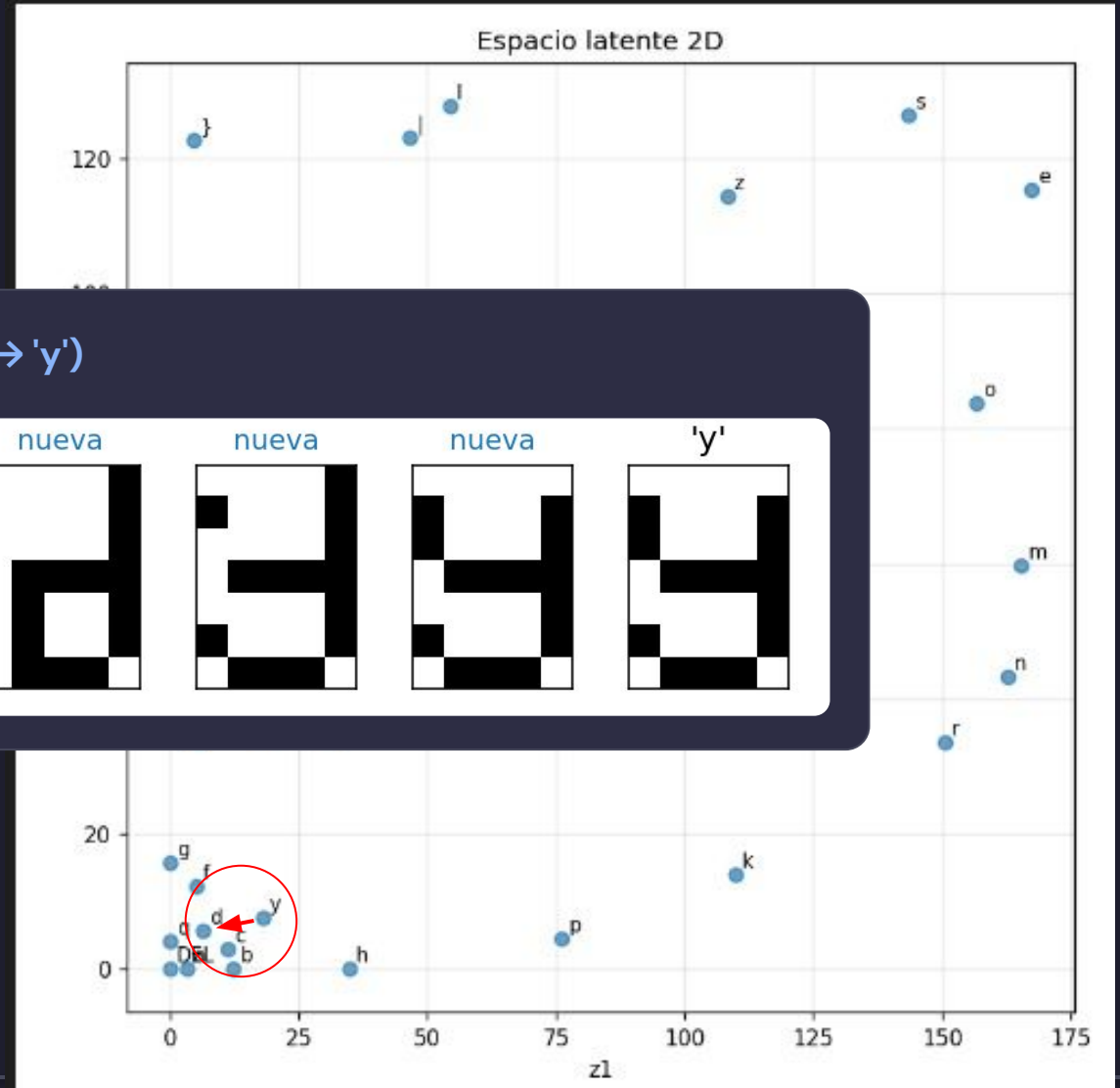
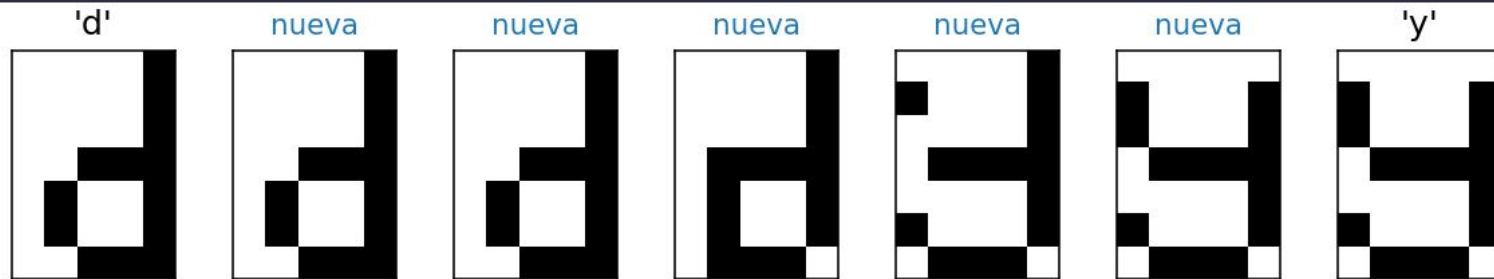
03. Espacio Latente y Generación



Generación de Letras Nuevas

03. Espacio Latente y Generación

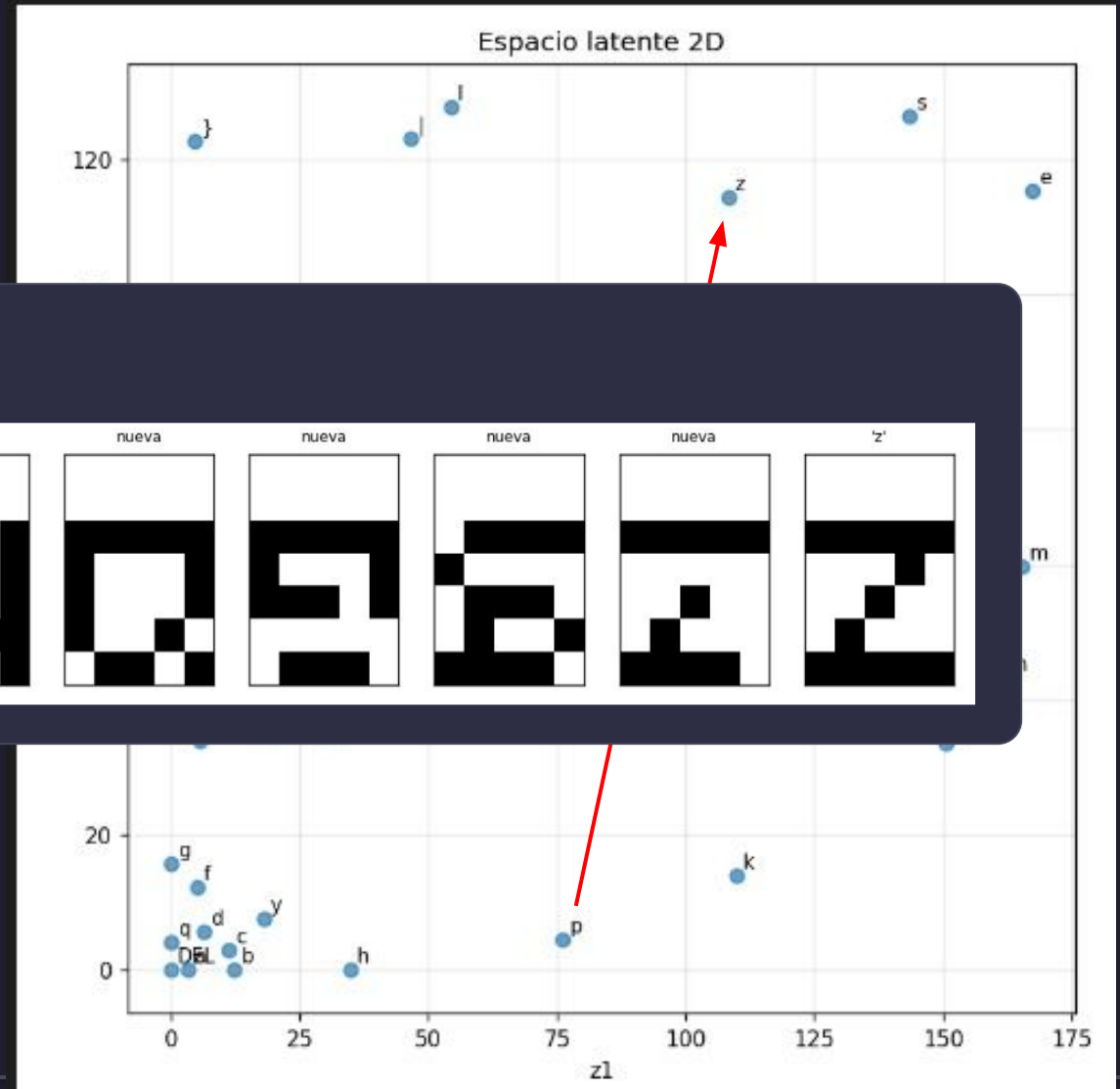
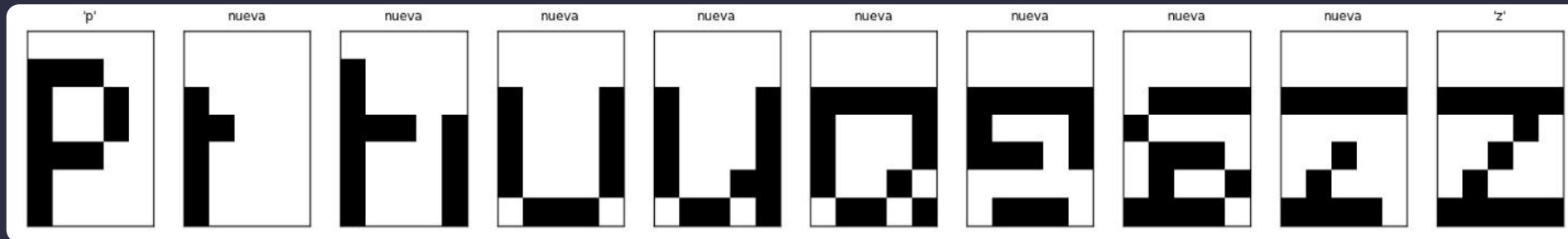
Interpolación: Caracteres Cercanos ('d' → 'y')



Generación de Letras Nuevas

03. Espacio Latente y Generación

Interpolación: Caracteres Distantes ('p' → 'z')



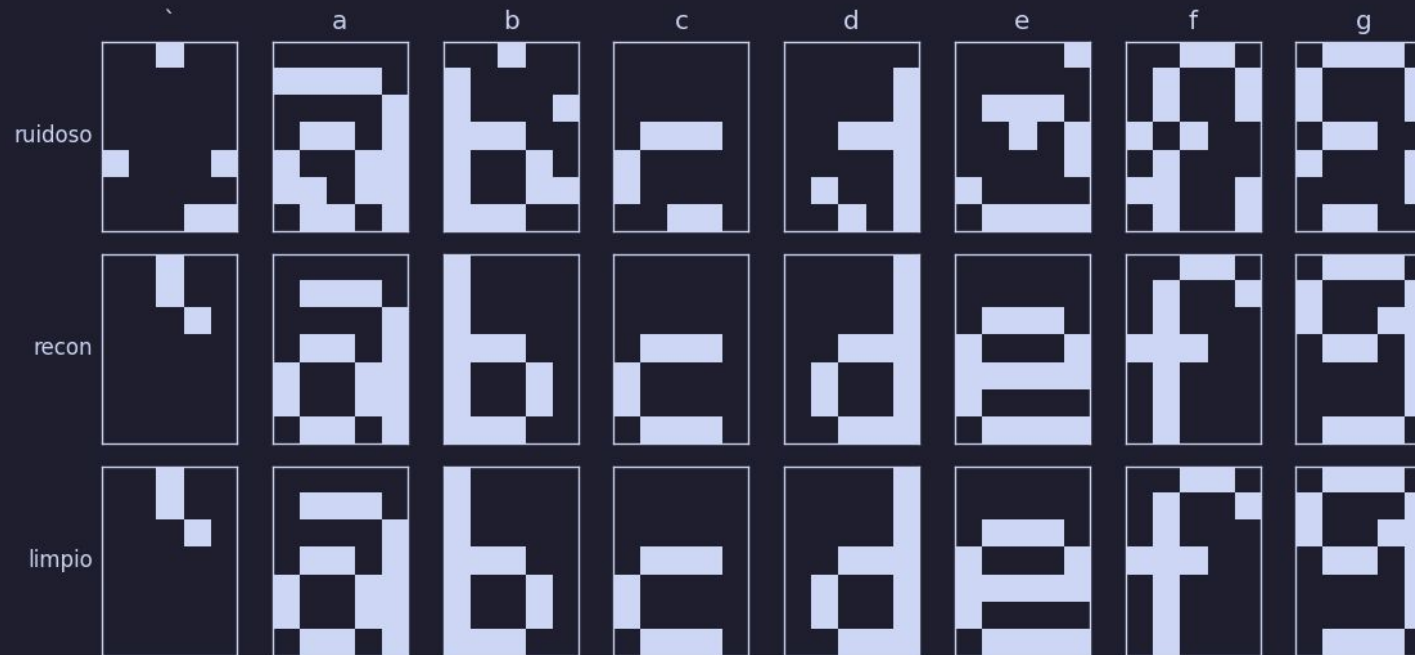
04

Denoising Autoencoder

Denoising: Tarea y Metodología

04. Denoising Autoencoder

- Tarea: reconstruir el glifo limpio desde una entrada corrupta - salt & pepper sobre los 35 pixeles
- Ruido nuevo en cada época.
- Latente 2D.



Búsqueda de Configuración

04. Denoising Autoencoder

Metodología de Búsqueda

Una variable a la vez: 15 configuraciones × 3 seeds diferentes, manteniendo la misma evaluación fija para asegurar consistencia.

Configuraciones Ganadoras

- Réplicas ruidosas ×5
- Función de activación leaky_relu
- Ruido fijo de 0.20

Elementos Descartados

No aportaron mejoras significativas: menor profundidad, decodificador asimétrico, reducción de épocas o uso de tanh en el espacio latente.

Insight Clave: El Cuello de Botella

La proyección del encoder es crítica: a mayor cantidad de corrupciones por paso, se logra una robustez superior en el modelo final.

Config Final: Resultados (5 Seeds)

04. Denoising Autoencoder

```
{
  "architecture": {
    "encoder_layers": [35,29,22,15,8,2],
    "activation": "leaky_relu",
    "latent_activation": "linear"
  },
  "training": {
    "epochs": 60000, "lr": 0.003,
    "lr_schedule": "cosine", "restarts": 10
  },
  "denoising": {
    "noise_type": "salt_pepper",
    "level": 0.2,
    "replicas": 5
  }
}
```

% de glifos perfectos por nivel de ruido (eval fija)

99.8%

ruido 0.05

99.0%

ruido 0.10

96.3%

ruido 0.20

89.0%

ruido 0.30

05

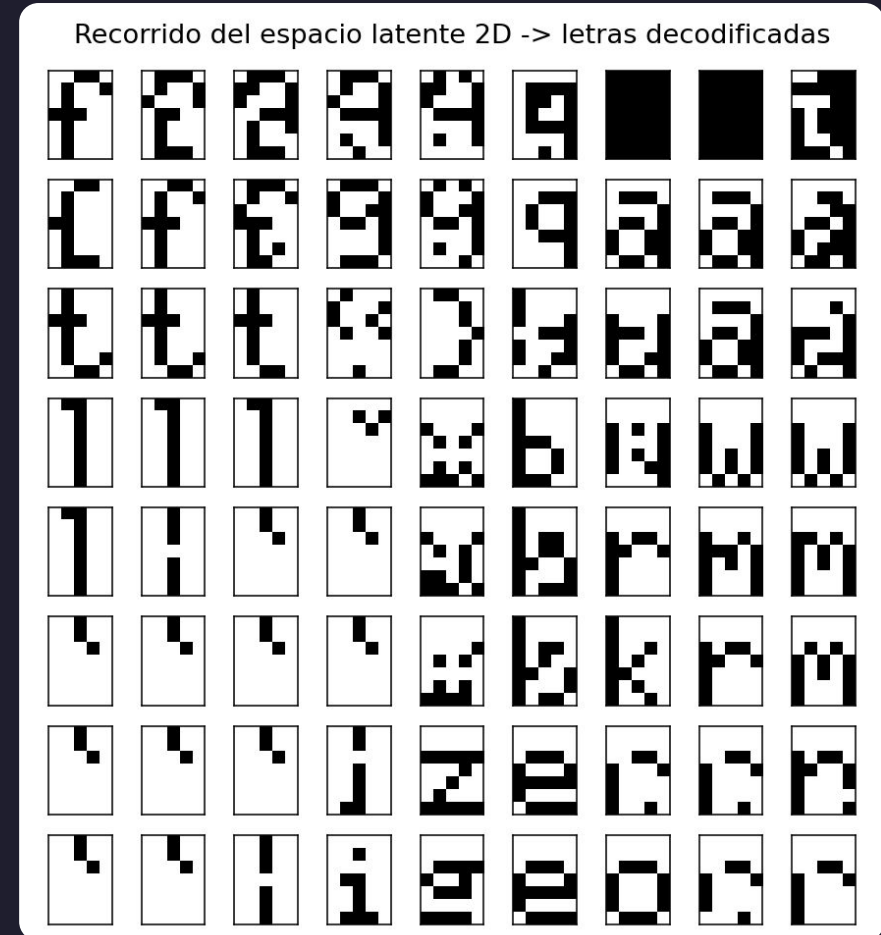
Variational Autoencoder

Limitaciones del AE simple

05. Variational Autoencoder

Problema de Generación

- El espacio latente es discontinuo, con huecos que impiden un muestreo coherente.
- Al muestrear puntos al azar, el decoder genera "basura".
- No existe una distribución real de la que pueda generar información nueva.



Espacio latente discontinuo → falla en la generación intermedia

Solución: VAE

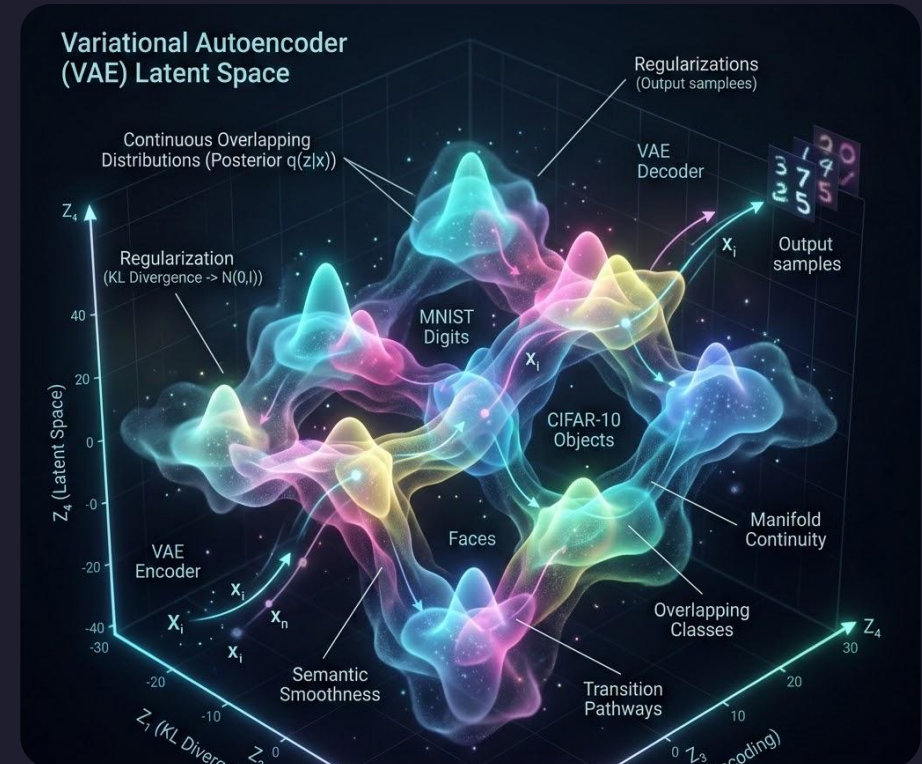
05. Variational Autoencoder

1 Encoder de Distribución

Devuelve una distribución (μ, σ) en lugar de un punto. Esto crea "blobs" que se solapan, asegurando continuidad y eliminando huecos en el espacio latente.

2 Regularización KL

Empuja el posterior hacia un prior $N(0, I)$. Retiene a las imágenes en una región ordenada y conocida, facilitando la generación posterior.



Resultado: Latente continuo y muestreable

Implementación

05. Variational Autoencoder

Encoder MLP

Capas densas + ReLU. No termina en un vector, sino en dos arreglos lineales: μ y $\log\sigma^2$ de dimensión latent_dim.

Reparametrización

Muestreo diferenciable: $z = \mu + e^{\{\log\sigma^2/2\}} \cdot \varepsilon$
Con $\varepsilon \sim N(0, I)$ muestreado fresco en cada paso.

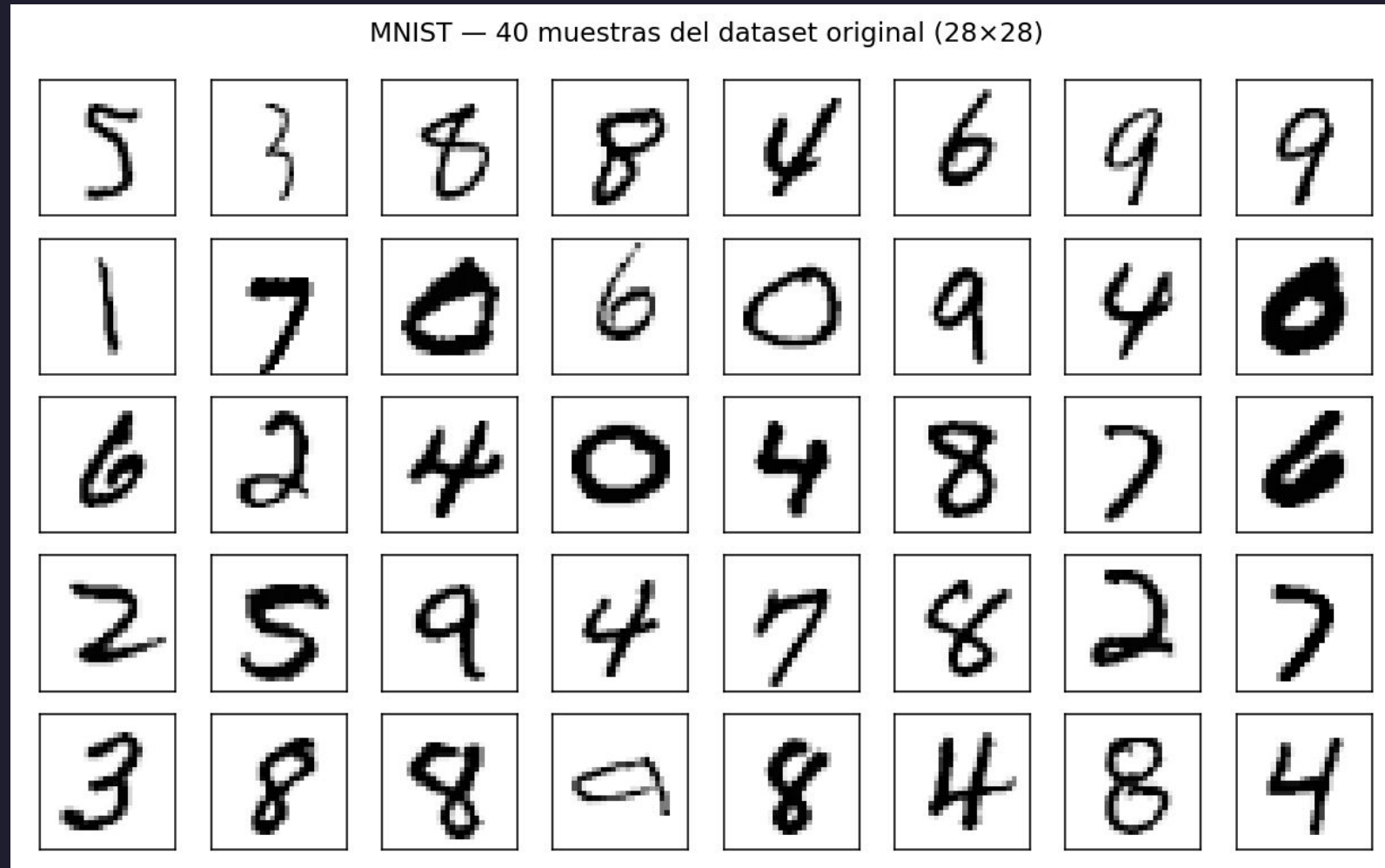
Función de Pérdida

ELBO = reconstrucción + $\beta \cdot \text{KL}$

```
"architecture": {  
  "encoder_layers": [784, 256, 64],  
  "latent_dim": 2,  
  "activation": "relu",  
  "output_activation": "sigmoid",  
  "init": "he_normal"  
},  
"training": {  
  "loss": "bce",  
  "epochs": 2000,  
  "lr": 1e-3,  
  "beta": 1.0,  
  "beta_warmup": 200,  
  "seed": 0,  
  "log_every": 100  
}
```

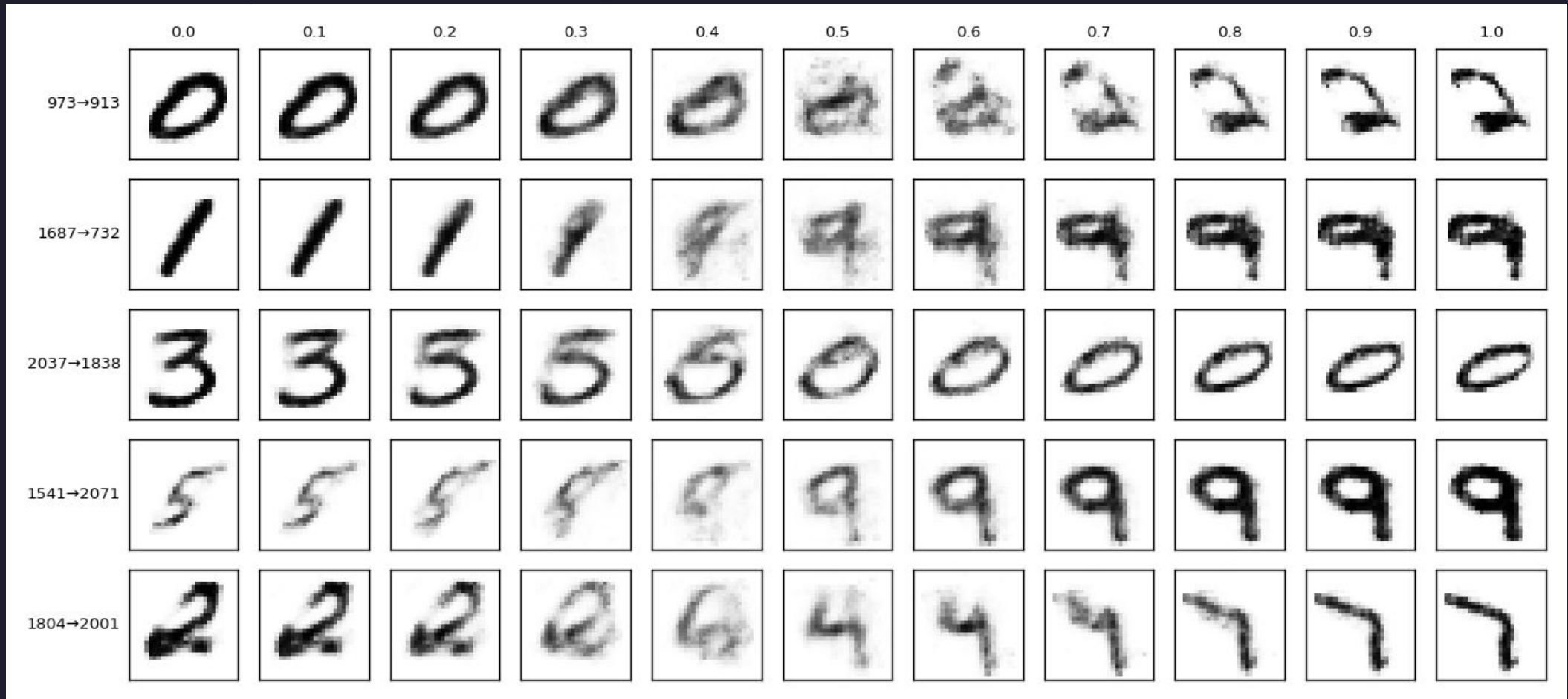
Dataset MNIST

05. Variational Autoencoder



¿Funciona?

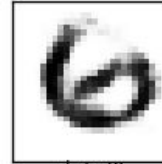
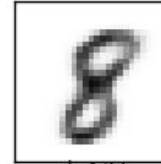
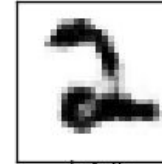
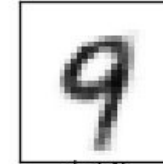
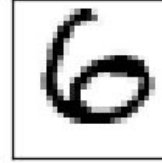
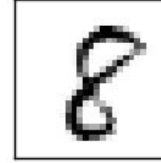
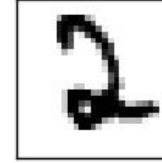
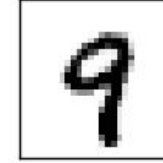
05. Variational Autoencoder



¿Funciona?

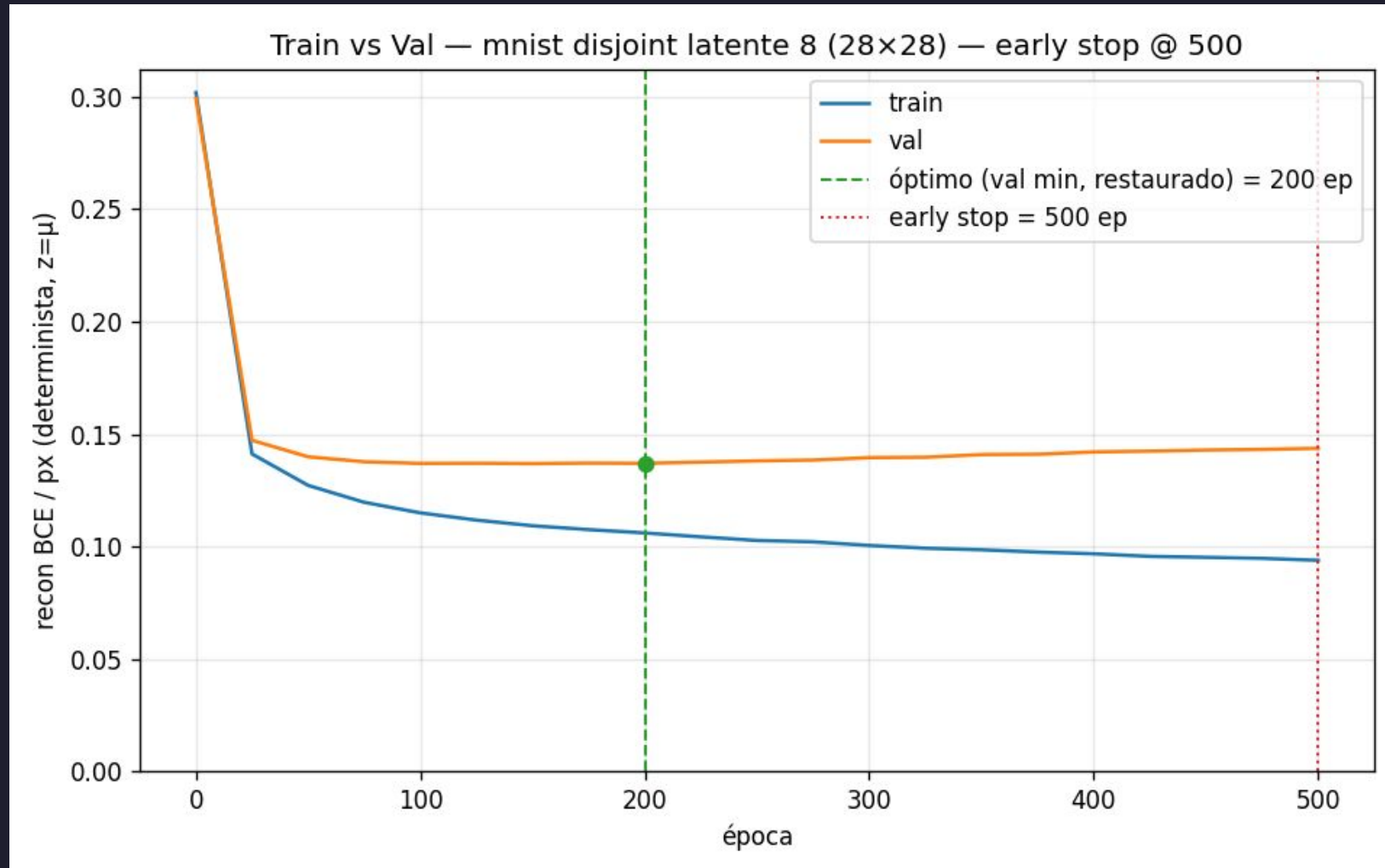
05. Variational Autoencoder

Muestras generadas vs vecino más cercano en train (L2 media = 4.37)

sample	 d=3.33	 d=5.12	 d=4.93	 d=3.36	 d=3.40	 d=5.70	 d=4.27	 d=4.75	 d=3.46	 d=5.55	 d=4.82	 d=3.75
vecino												

¿Funciona?

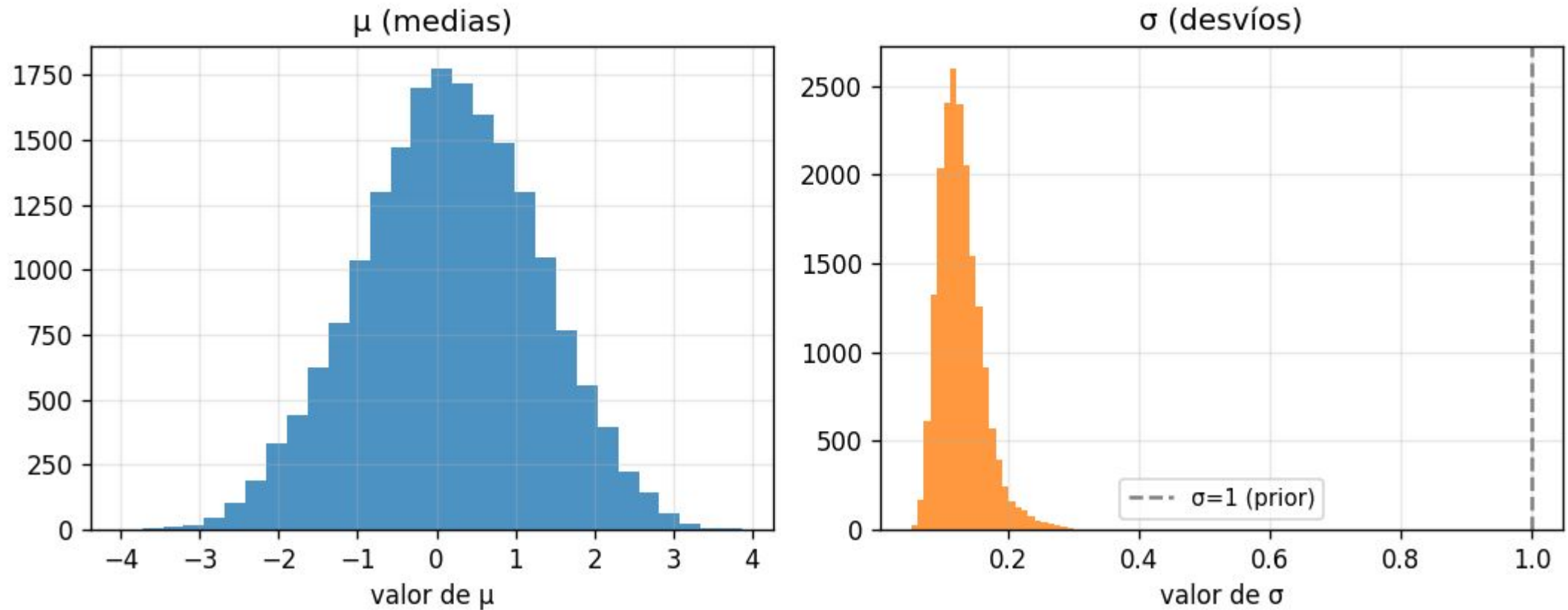
05. Variational Autoencoder



¿Funciona?

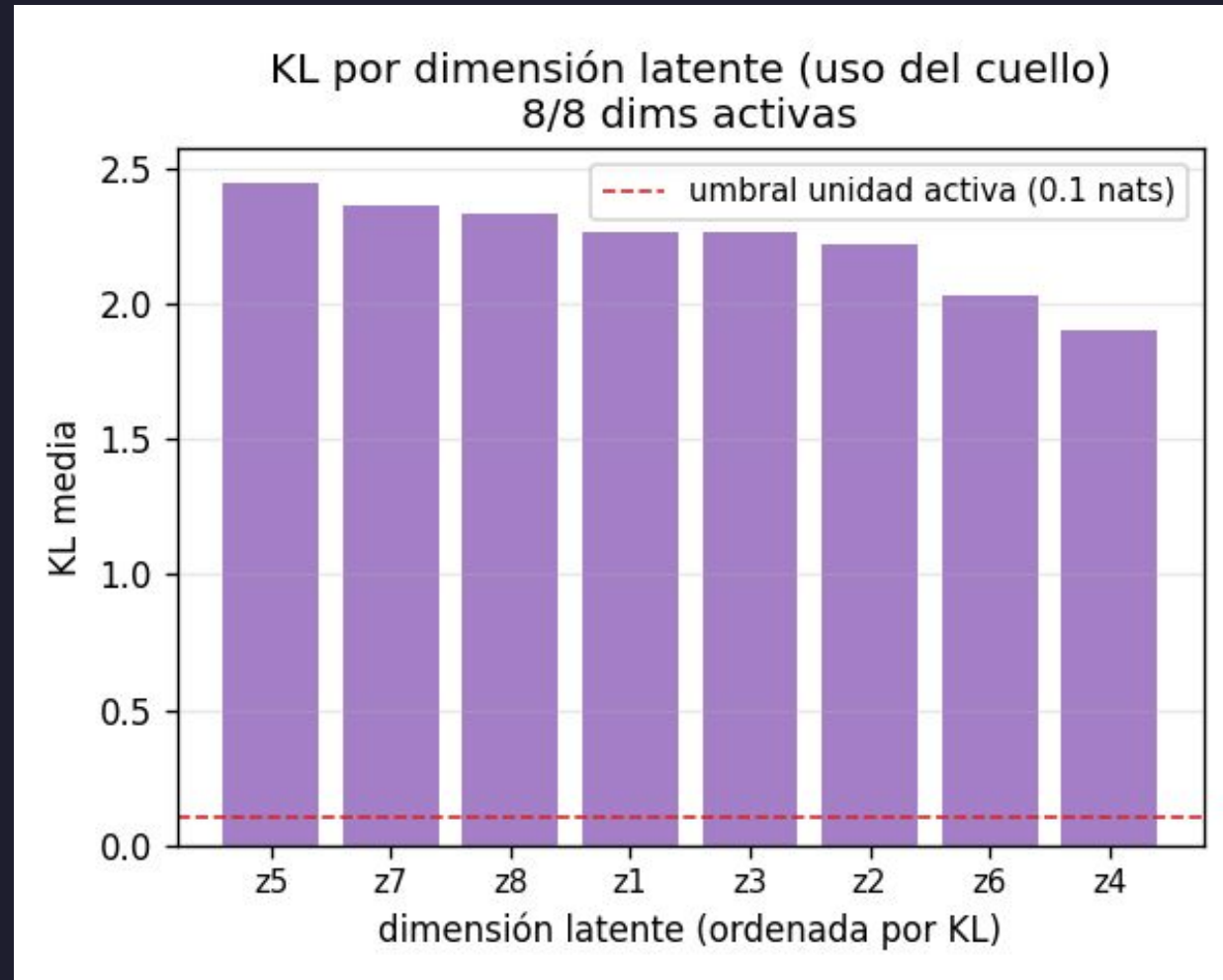
05. Variational Autoencoder

Distribución del posterior $q(z|x)$



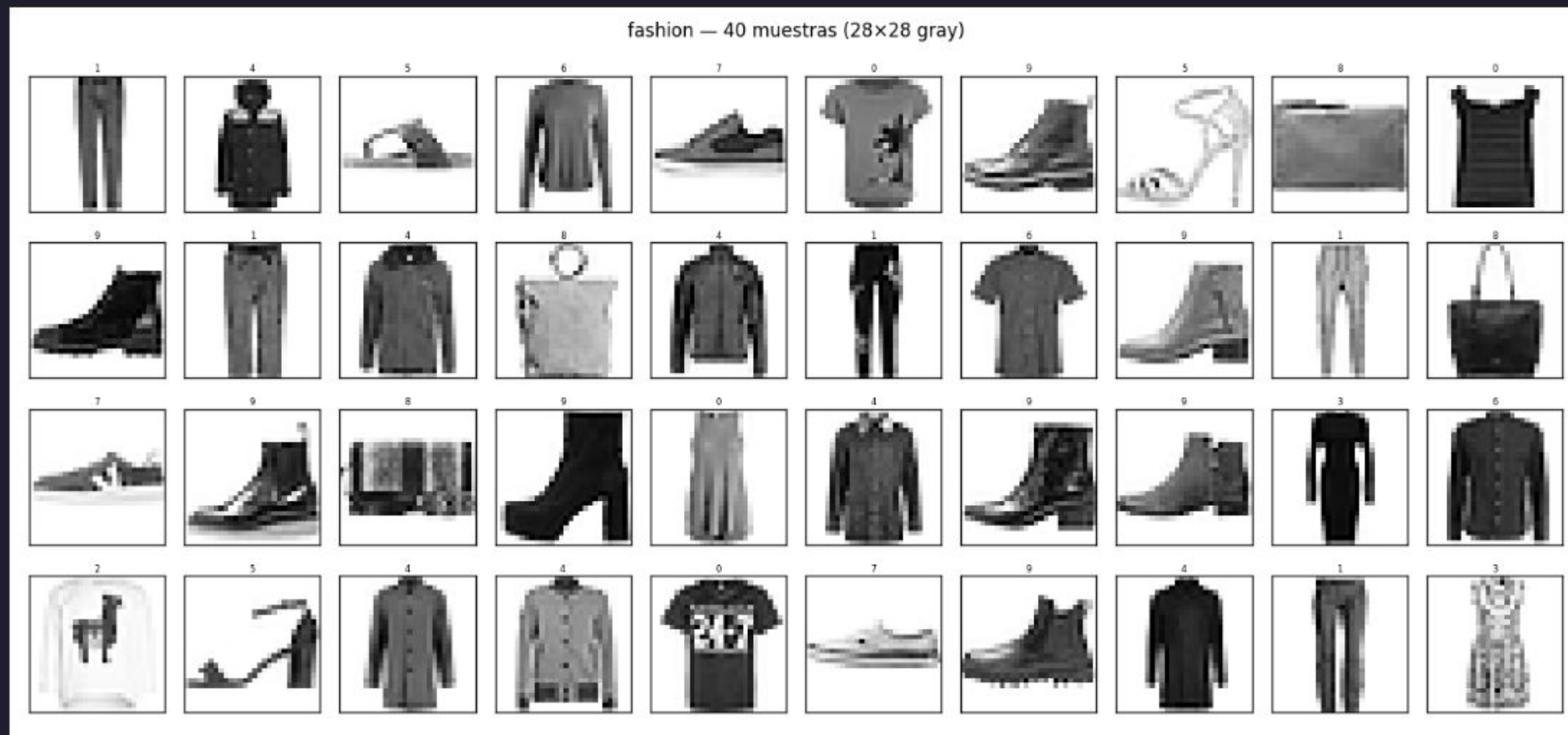
¿Funciona?

05. Variational Autoencoder



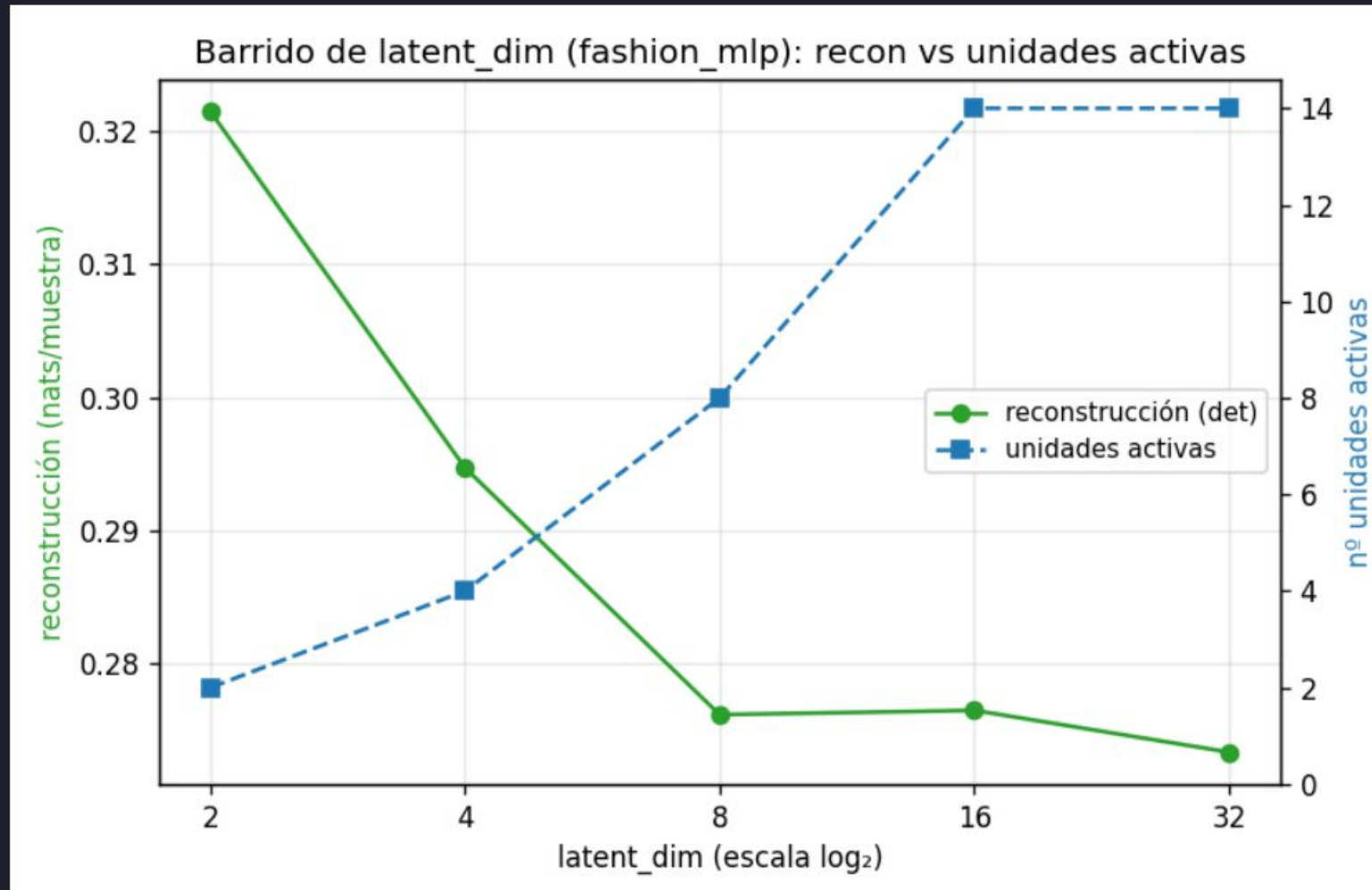
Dataset fashion

05. Variational Autoencoder



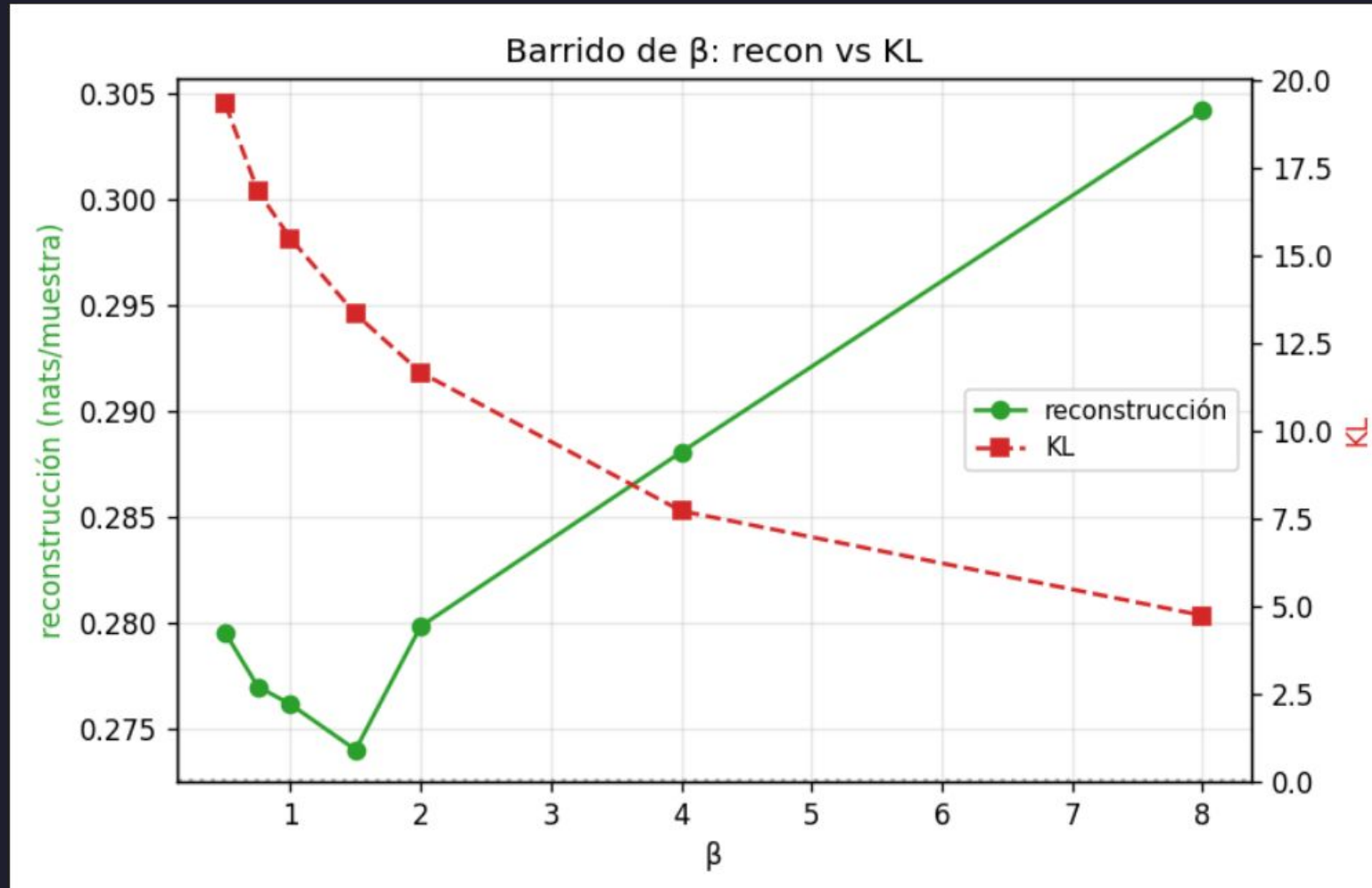
Mejor configuración: latente

05. Variational Autoencoder



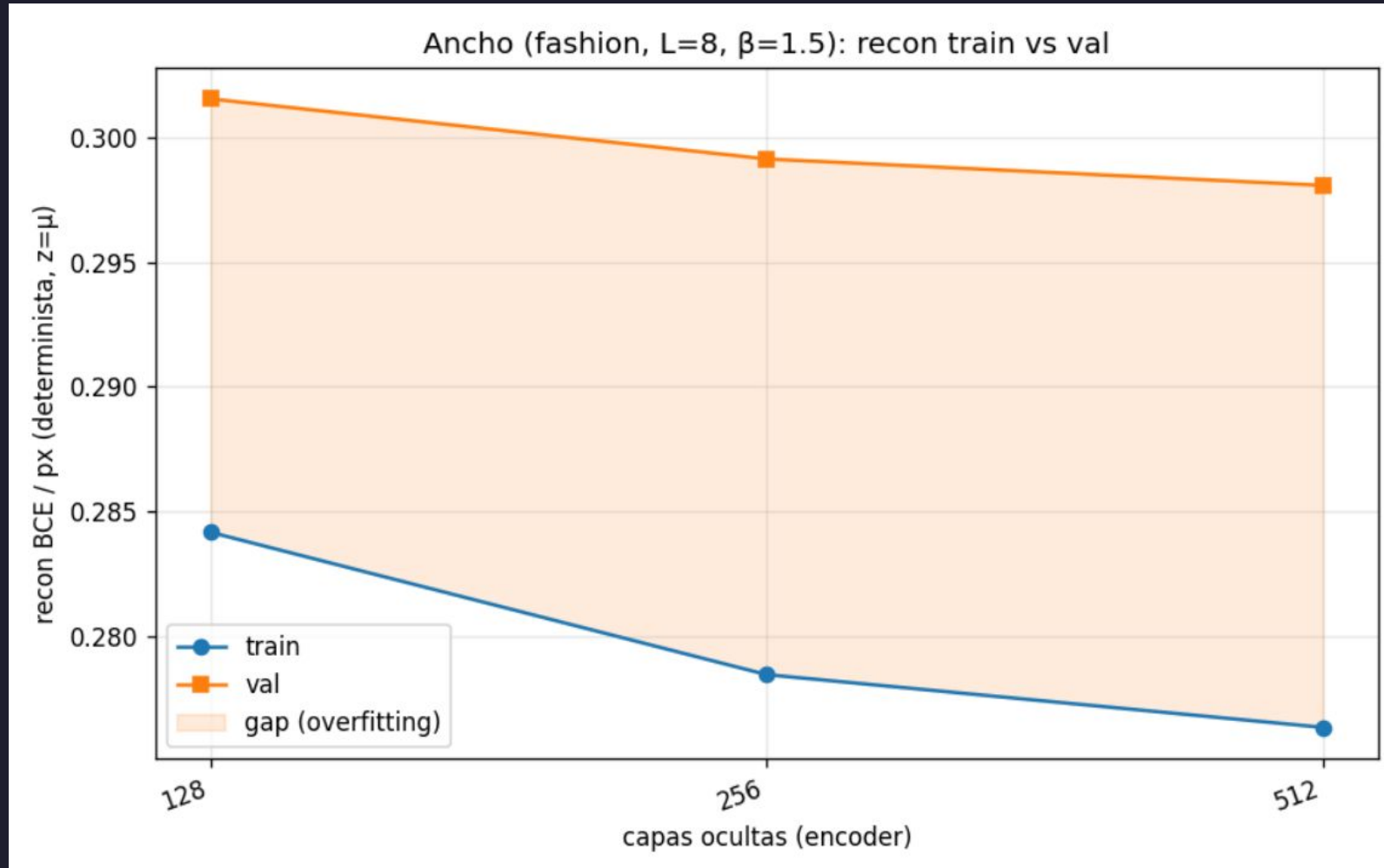
Mejor configuración: β

05. Variational Autoencoder



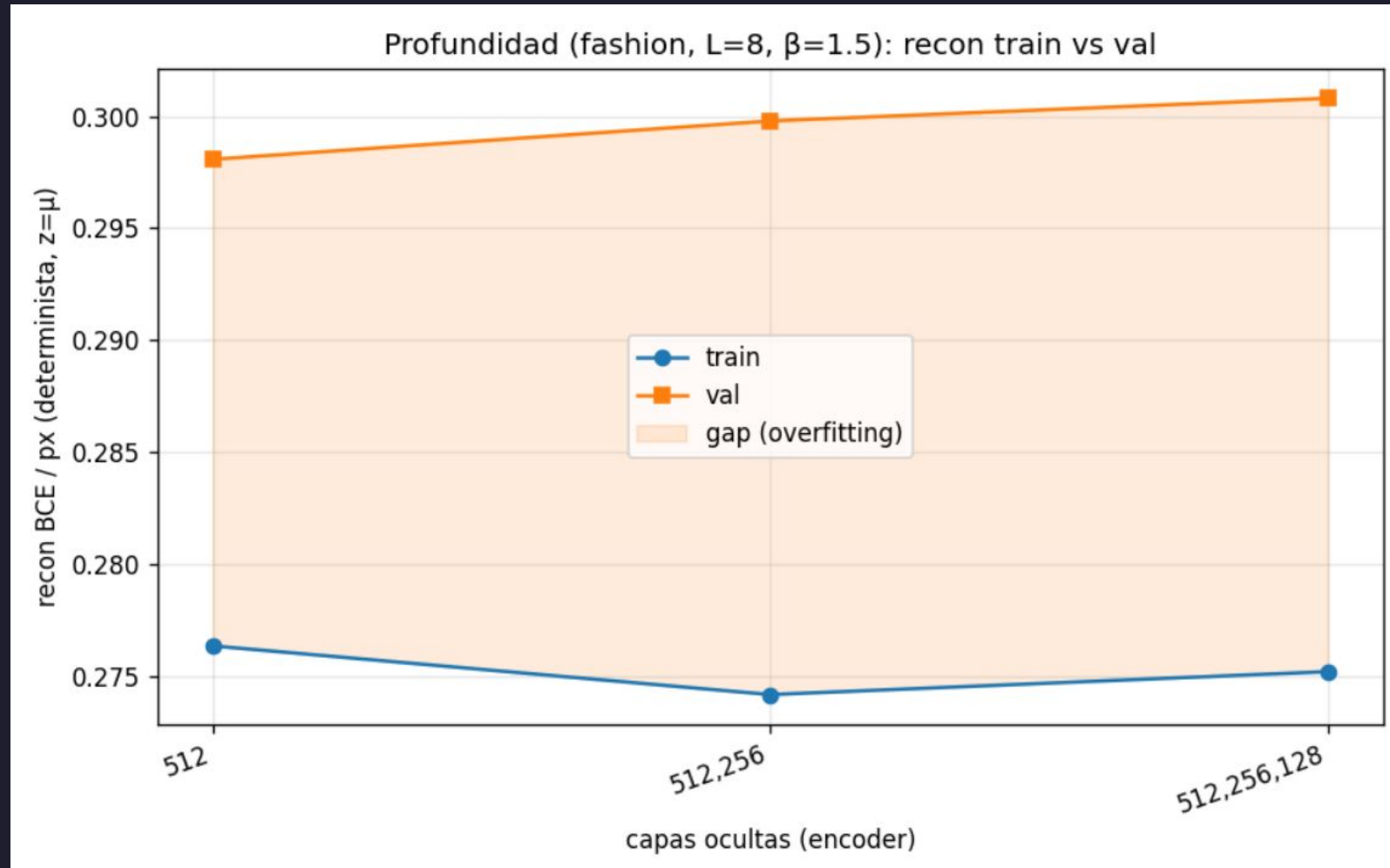
Mejor configuración: arquitectura

05. Variational Autoencoder



Mejor configuración: arquitectura

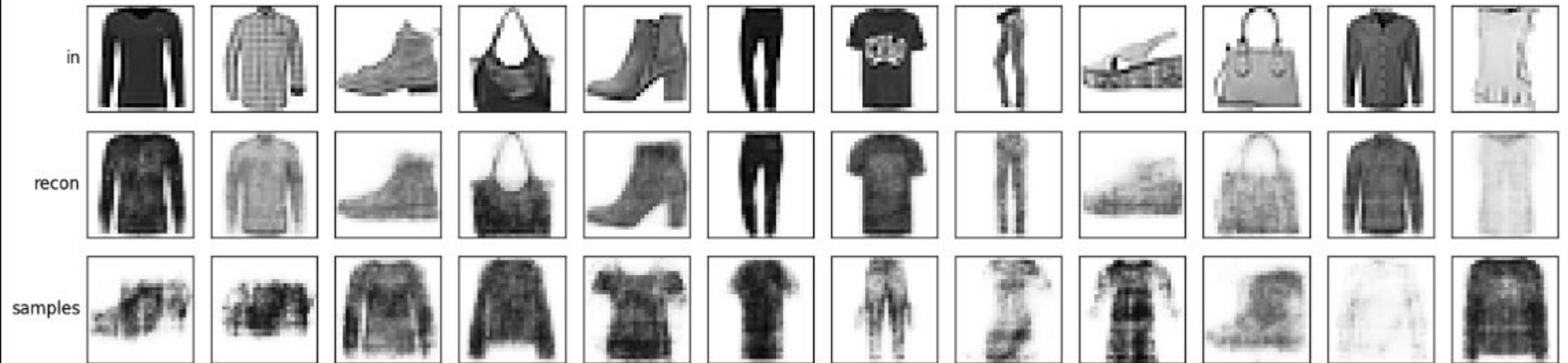
05. Variational Autoencoder



Resultado – Recon, ok; ¿generación?

05. Variational Autoencoder

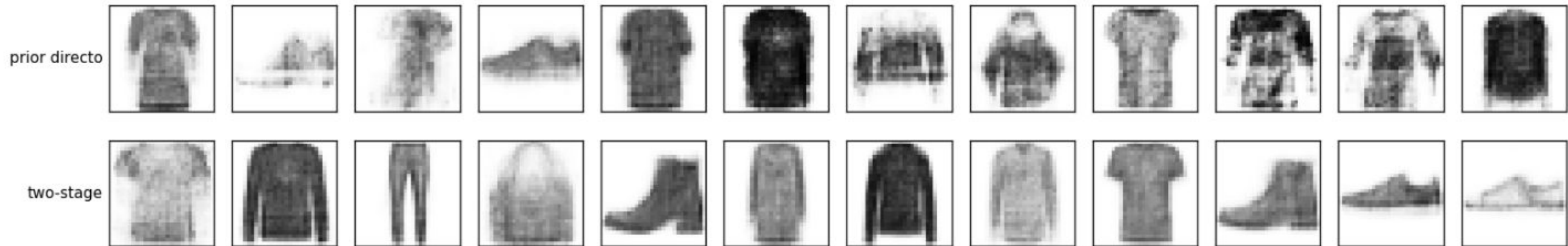
VAE — fashion latente 8 h[512] (28x28), recon TRAIN + samples (prior directo)



Optimización de gen: two-stage VAE

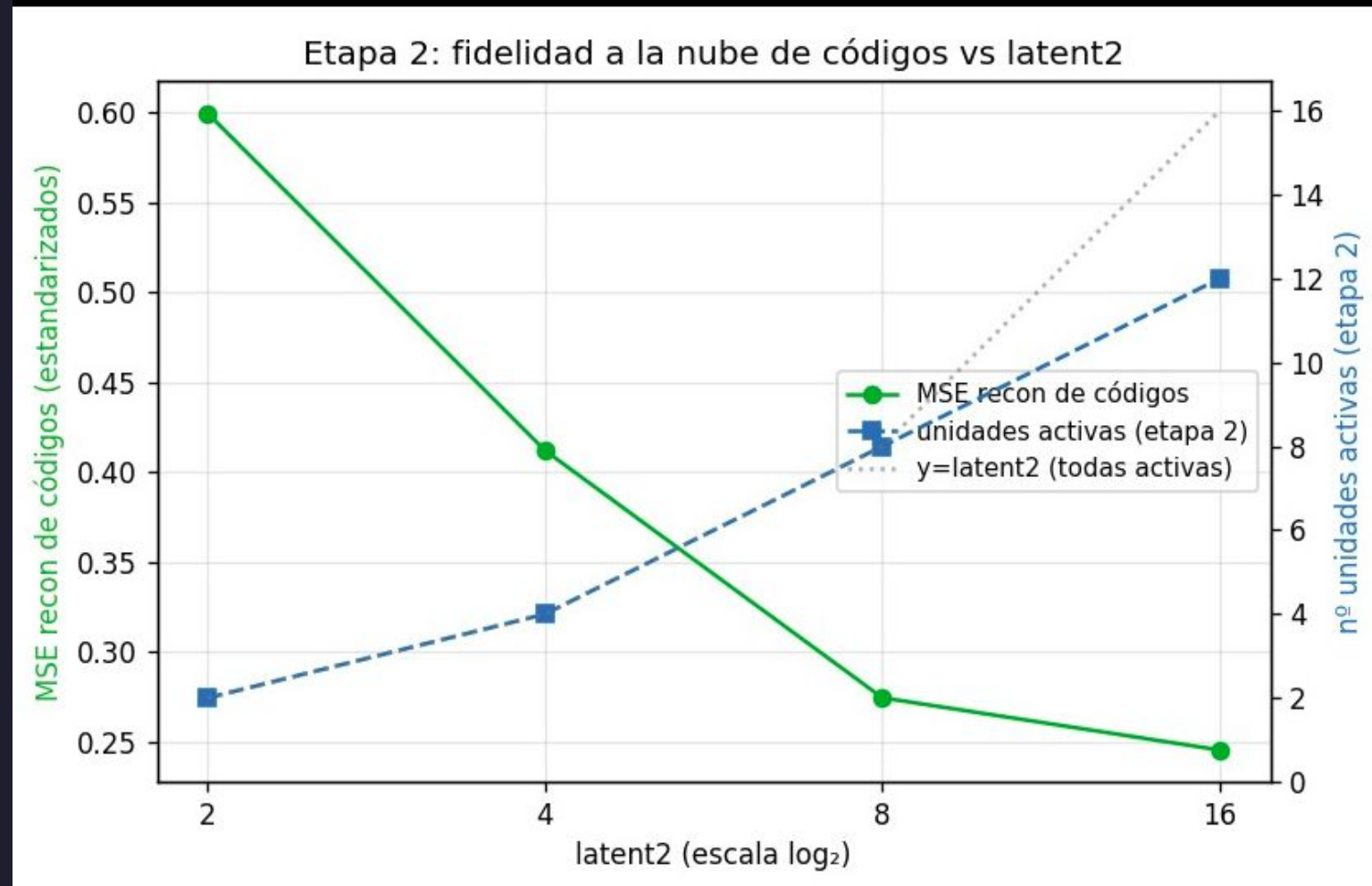
05. Variational Autoencoder

Prior directo vs Two-stage — fashion



Barrido latent2: variabilidad

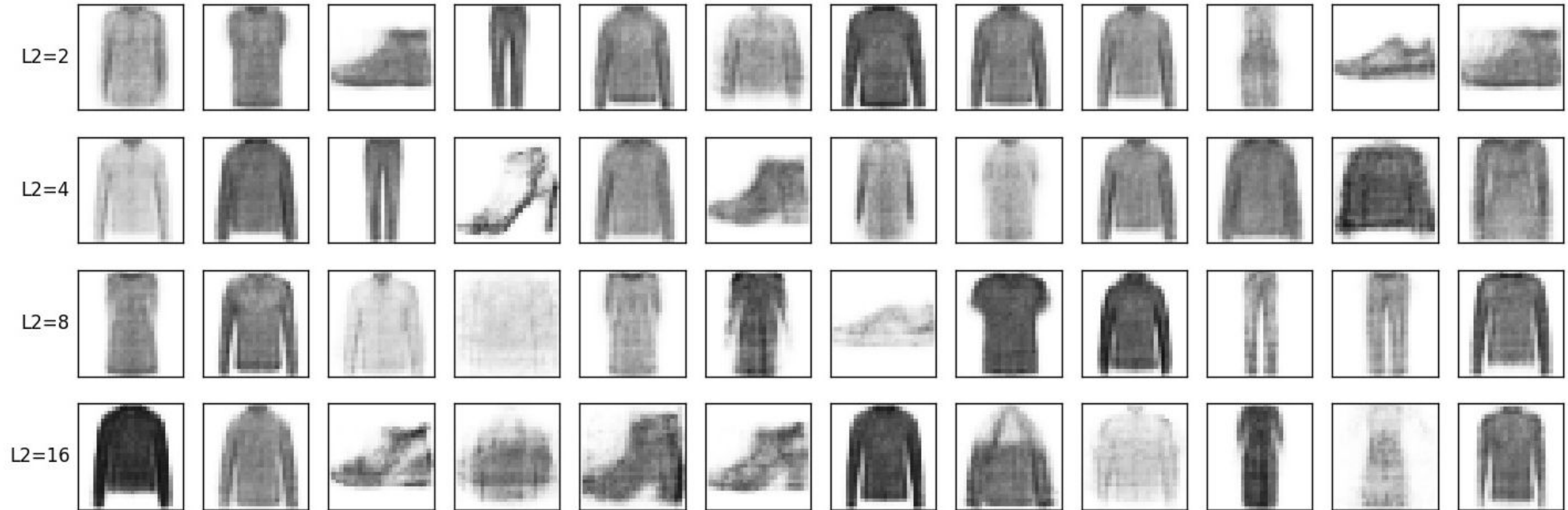
05. Variational Autoencoder



Barrido latent2: variabilidad

05. Variational Autoencoder

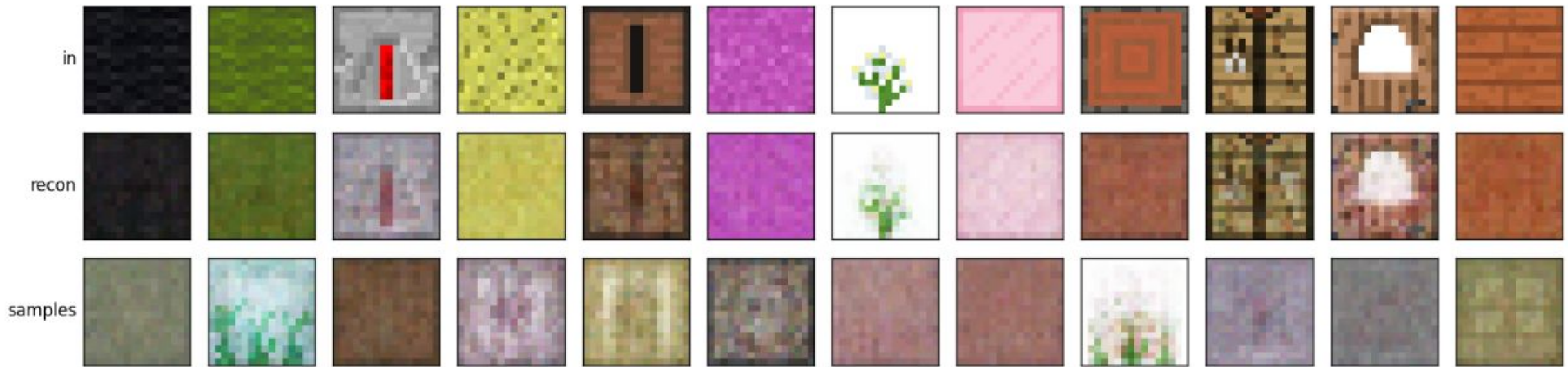
Muestras generadas vs latent2 (etapa 1 fija) — afinan y saturan al subir L2



Otros datasets: Minecraft

05. Variational Autoencoder

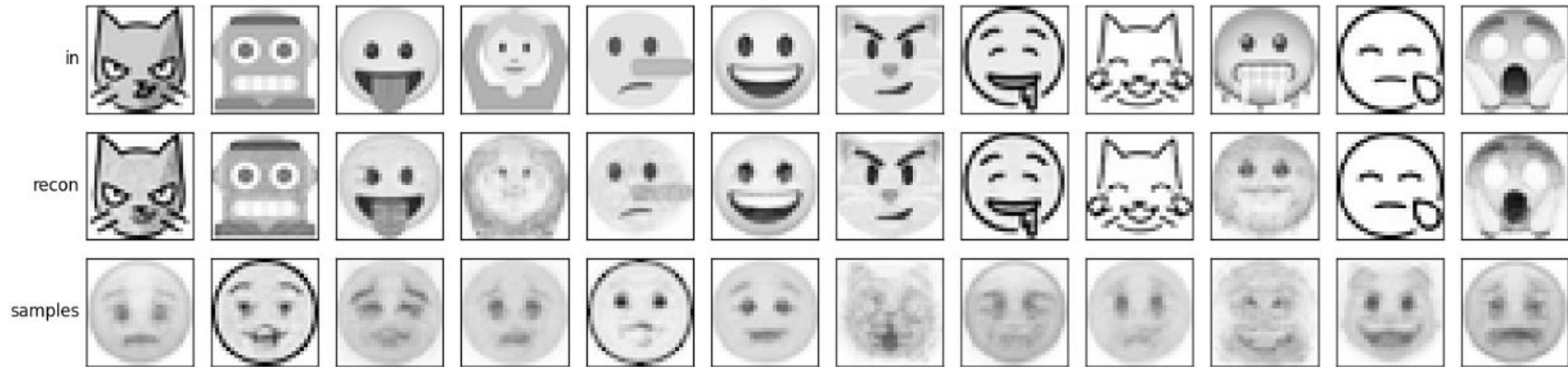
VAE — minecraft-old latente 32 h[512] (16×16), recon TRAIN + samples two-stage (L2=32)



Otros datasets: Emojis

05. Variational Autoencoder

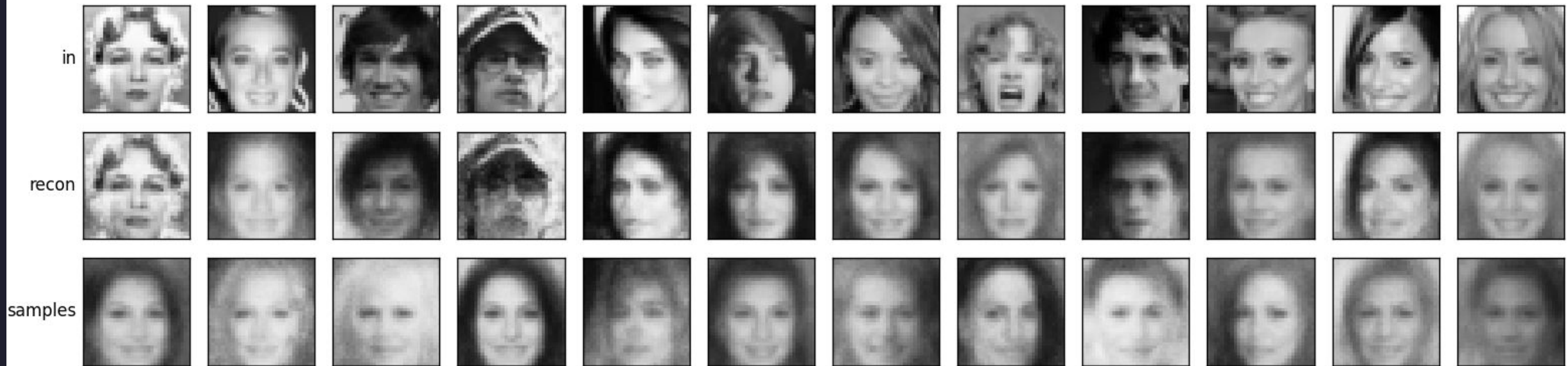
VAE — emoji_multi latente 16 h[512,256] (28×28), recon TRAIN + samples two-stage (L2=8)



Otros datasets: CelebA

05. Variational Autoencoder

VAE — celeba latente 16 h[512,256] (32×32), recon TRAIN + samples two-stage (L2=12)



06

Conclusiones

Conclusiones Finales

06. Conclusiones



Kohonen – Radio de vecindad

Trade-off entre precisión local y topología global.



Oja – Regla de Aprendizaje

Para $LR > 0.5$, permite calcular con exactitud la PC1 de los datos.



VAE vs. AE simple

El VAE no mejora la reconstrucción, pero aporta un latente estructurado y generativo.



Modelo de Hopfield

La ortogonalidad de patrones es tan importante como el tamaño de la red.

Muchas Gracias

¿Preguntas?